SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Hoang-Nam Tran

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Performance prediction of iterator based database systems with decision trees

Leistungsprognose iteratorbasierter Datenbanksysteme mit Entscheidungsbäumen

Author:	Hoang-Nam Tran
Examiner:	Prof. Dr. Thomas Neumann
Supervisor:	Maximilian Rieger, M.Sc.
Submission Date:	27.04.2025

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 27.04.2025 Hoang-Nam Tran

Acknowledgments

Abstract

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Section 1
1.1.1 Subsection 1

2 Theoretical Background **3**
2.1 Database Basics 3
2.2 Performance Prediction 3
2.3 Benchmarking 3
2.4 Cost Models 3
2.5 Decision Trees 3
2.6 PostgreSQL 3
2.7 Umbra 3
2.8 T3 3
2.9 Related Work 3
2.10 Section 3
2.10.1 Subsection 4

3 Methodology and Implementation **6**
3.1 Problem Statement and Prediction Target 6
3.2 Data and Preprocessing 6
3.3 Source Model 7
3.4 System A: Direct T3 Transfer Baseline 8
3.5 System B: T3 Transfer with PostgreSQL-native Features 8
3.6 Training Setup 8
3.7 Evaluation Metrics 8
3.8 Section 8
3.8.1 Subsection 8

4 Results **10**
4.1 Section 10
4.1.1 Subsection 10

5 Discussion **12**
5.1 Section 12
5.1.1 Subsection 12

6 Conclusion **14**
6.1 Section 14
6.1.1 Subsection 14

Abbreviations **16**

List of Figures **17**

List of Tables **18**

Bibliography **19**

1 Introduction

1.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}`, `\ac{TUM}` ⇒ Technical University of Munich (TUM), TUM

For more details, see the documentation of the `acronym` package[1].

1.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 1.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

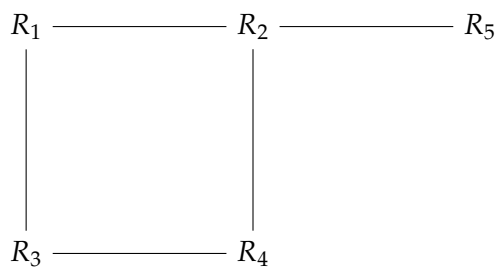

Figure 1.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

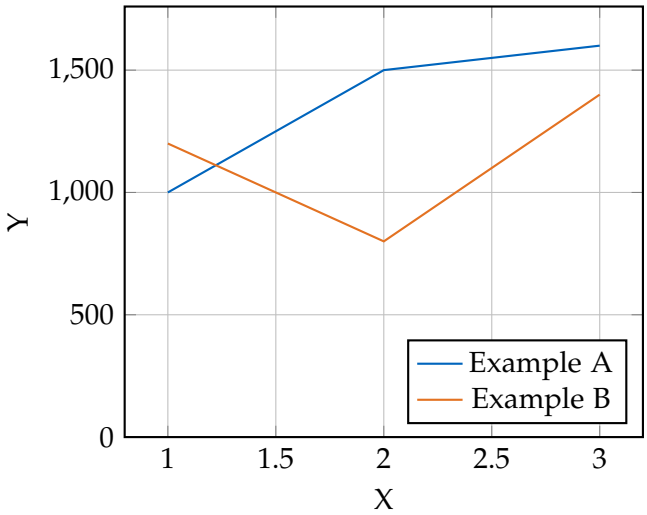

Figure 1.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3: An example for a source code listing.

2 Theoretical Background

2.1 Database Basics

operator type, estimated/actual cardinalities, widths, and runtimes

2.2 Performance Prediction

draft: beschreiben, was performance prediction ist, was die ziele sind, etc. benchmarking, etc. Cost Model beschreiben

2.3 Benchmarking

timed queries, etc.

2.4 Cost Models

2.5 Decision Trees

2.6 PostgreSQL

2.7 Umbra

2.8 T3

2.9 Related Work

2.10 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

2.10.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 2.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

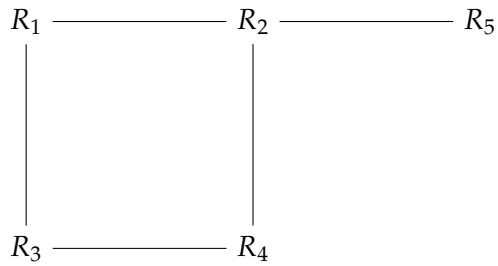

Figure 2.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

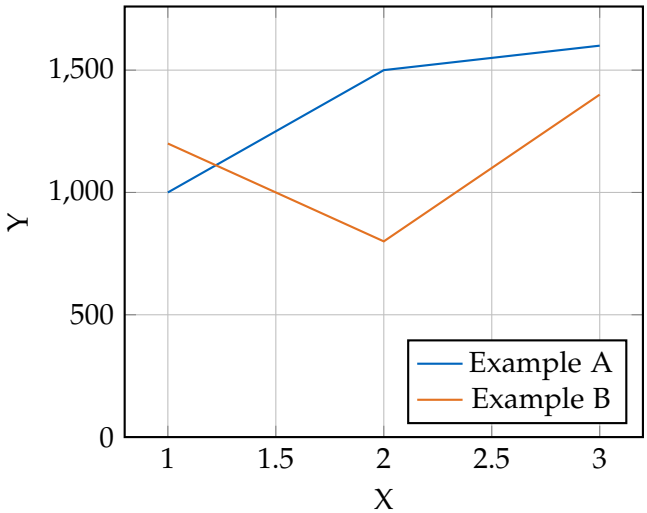

Figure 2.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 2.3: An example for a source code listing.

3 Methodology and Implementation

3.1 Problem Statement and Prediction Target

In this thesis, we address the transfer of the T3 runtime prediction approach from Umbra to PostgreSQL parsed plans. The central methodological goal is to preserve T3's core modeling principles. Upon that, we adapt plan conversion and feature extraction to PostgreSQL-specific plan information.

Also, the prediction target is total query runtime. Following T3, each query plan is decomposed into pipelines and a per-tuple runtime is predicted for each pipeline. Then, we aggregate the per-tuple and pipeline runtimes to obtain the final query-runtime estimate.

3.2 Data and Preprocessing

For data collection, we use the public Zero-Shot benchmark data introduced with the Zero-Shot Cost Model [HB22]. The benchmark is particularly suitable for this thesis because it combines real-world datasets with established benchmark workloads. It combines real-world data (e.g., IMDB with JOB queries) with synthetic benchmark data from SSB and TPC-H. Overall, the benchmark contains 20 databases with large variation in tables, columns, and foreign-key relationships.

Having multiple databases allows us to test the generalization capabilities of our model. Similar to the T3 paper, we can train the model on all but one databases and test on the remaining one. Moreover as the widely used JOB workload is included, we can compare our performance predictions to other implementations later on. Further, PostgreSQL is the intended database system of our implementation, which avoids the effort of building a separate PostgreSQL benchmark.

Following the benchmark construction described in the paper, queries are generated per database and executed on a PostgreSQL DBMS (v12) on c8220 nodes on the CloudLab platform. Then, they are stored as raw traces (i. e., the output of `EXPLAIN ANALYZE`). Next, the authors preprocess the queries to remove very long-running queries (queries above 30 s are excluded) to keep the benchmark practical and reproducible [HB22]. A further preprocessing step is to parse the raw traces into structured plan

representations. These parsed plans (15000 per database) contain the operator trees with node-wise attributes (e.g., operator type, estimated/actual cardinalities, widths, and runtimes).

After preprocessing, we get plans that are similar to the Umbra plan structure, since they also consist of operator trees. Hence, the parsed plans offer a good starting point for our pipeline and we can begin without further preprocessing.

3.3 Source Model

T3 is our source model and we transfer its core prediction logic from Umbra to PostgreSQL parsed plans. Hence, we adopt its three central design principles: A pipeline-based query plan representation, tuple-centric prediction targets, and gradient-boosted decision trees for runtime prediction. [RN25]

In our implementation, we preserve these principles and adapt only representation-specific components. First, PostgreSQL parsed plans are converted into a T3-compatible internal structure in `src/zeroshot/zeroshot_to_t3.py`. This conversion includes operator mapping, pipeline assignment, and extraction of plan-level metadata. Second, model training follows tuple-centric T3 logic in `src/zeroshot/training_zeroshot_tpch_holdout.py`: per-pipeline and per-tuple training data is generated, targets are transformed as $-\log(t)$, and a LightGBM model is trained with a fixed protocol.

To realize the transfer, we use two feature-design variants under the same learning setup:

1. **Direct transfer baseline:** reuse the existing Umbra-oriented `FeatureMapper` (`src/features.py`) on converted plans.

2. **PostgreSQL-native transfer variant:** use `PgFeatureMapper` (`src/pg_features.py`), which computes pipeline-level vectors directly from PostgreSQL plan payload (`pg/plan_parameters`) while preserving the same model interface and training flow.

The transfer principle is therefore the following: keep T3's learning and prediction mechanics unchanged, and adapt only plan conversion and feature representation where Umbra- and PostgreSQL-specific semantics differ.

3.4 System A: Direct T3 Transfer Baseline

3.5 System B: T3 Transfer with PostgreSQL-native Features

3.6 Training Setup

3.7 Evaluation Metrics

3.8 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

3.8.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 3.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

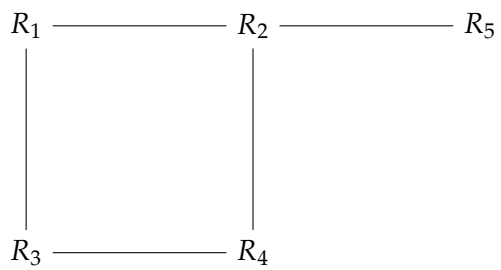

Figure 3.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

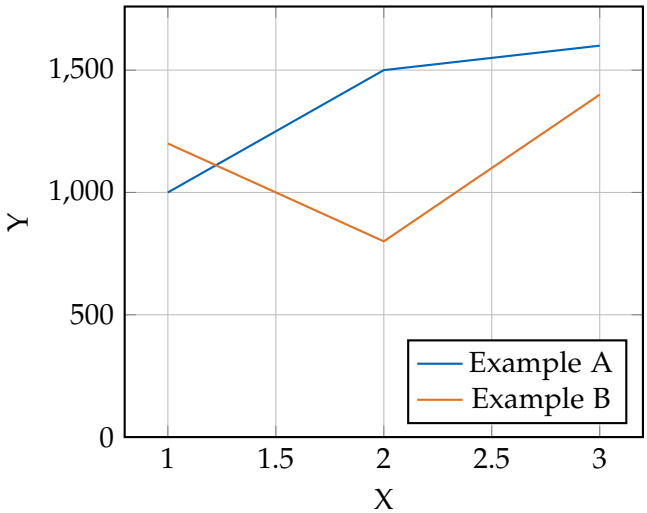

Figure 3.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 3.3: An example for a source code listing.

4 Results

4.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

4.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 4.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

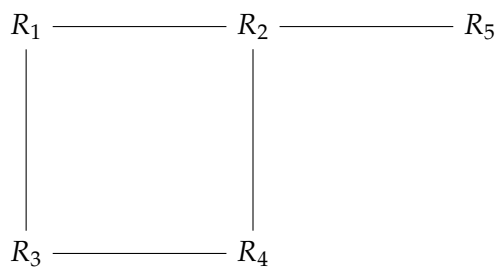

Figure 4.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

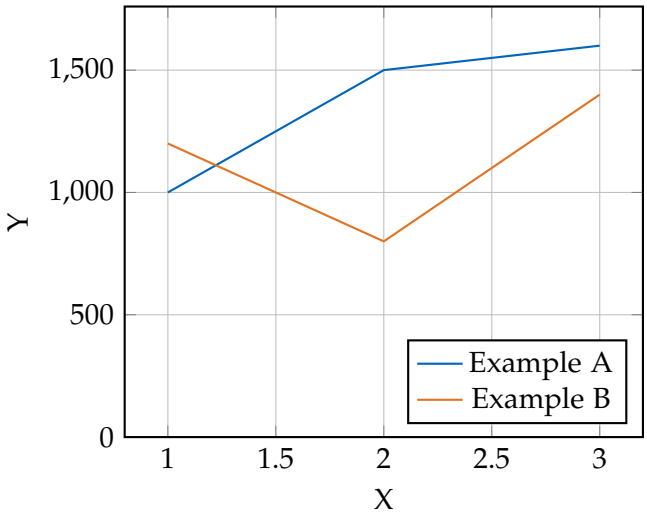

Figure 4.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 4.3: An example for a source code listing.

5 Discussion

Data from Zeroshot kann limitierender faktor sein.

5.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

5.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 5.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

[1] `https://ctan.org/pkg/acronym`

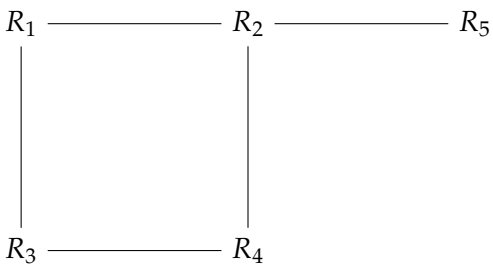

Figure 5.1: An example for a simple drawing.

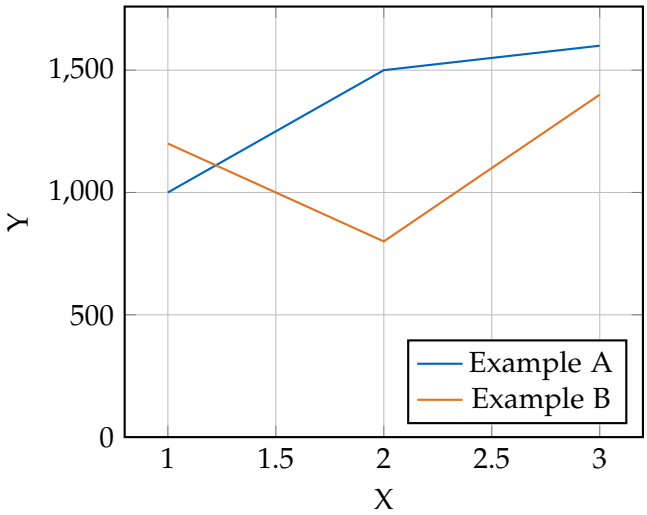

Figure 5.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 5.3: An example for a source code listing.

6 Conclusion

6.1 Section

Citation test [Lam94].

Acronyms must be added in `main.tex` and are referenced using macros. The first occurrence is automatically replaced with the long version of the acronym, while all subsequent usages use the abbreviation.

E.g. `\ac{TUM}, \ac{TUM}` ⇒ TUM, TUM

For more details, see the documentation of the `acronym` package[1].

6.1.1 Subsection

See Table 6.1, Figure 6.1, Figure 6.2, Figure 6.3.

Table 6.1: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

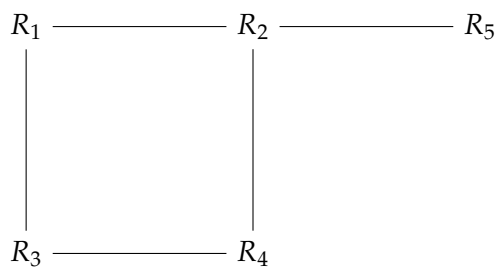

Figure 6.1: An example for a simple drawing.

[1] `https://ctan.org/pkg/acronym`

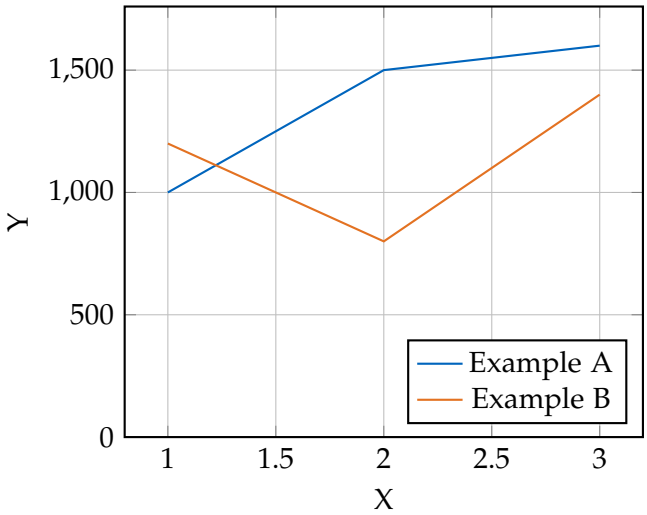

Figure 6.2: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 6.3: An example for a source code listing.

Abbreviations

TUM Technical University of Munich

List of Figures

1.1 Example drawing 1
1.2 Example plot 2
1.3 Example listing 2

2.1 Example drawing 4
2.2 Example plot 5
2.3 Example listing 5

3.1 Example drawing 8
3.2 Example plot 9
3.3 Example listing 9

4.1 Example drawing 10
4.2 Example plot 11
4.3 Example listing 11

5.1 Example drawing 13
5.2 Example plot 13
5.3 Example listing 13

6.1 Example drawing 14
6.2 Example plot 15
6.3 Example listing 15

List of Tables

1.1 Example table 1

2.1 Example table 4

3.1 Example table 8

4.1 Example table 10

5.1 Example table 12

6.1 Example table 14

Bibliography

[HB22] B. Hilprecht and C. Binnig. "Zero-shot cost models for out-of-the-box learned cost prediction." In: *Proc. VLDB Endow.* 15.11 (July 2022), pp. 2361–2374. ISSN: 2150-8097. DOI: 10.14778/3551793.3551799.

[Lam94] L. Lamport. *LaTeX : A Documentation Preparation System User's Guide and Reference Manual*. Addison-Wesley Professional, 1994.

[RN25] M. Rieger and T. Neumann. "T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees." In: *Proc. ACM Manag. Data* 3.3 (June 2025). DOI: 10.1145/3725364.